Institut Polytechnique de l'Université de Nice

Departement Mathématiques Appliquées et Modélisation (MAM)

Rapport Projet OptimalControl

Amiel Metier

Hedi Chennoufi

Supervisor : Jean-Baptiste Caillau, Oliver Cots

Rapport présenté en vue de satisfaire aux exigences de l'Université de Nice
pour l'obtention du diplôme de
Master of Science en Mathématiques Appliquées et Modélisation

17 mars 2026

Table des matières

1 Introduction **2**
1.1 Contexte global : Control Toolbox 2
1.2 Le module CTBenchmarks 2
1.3 Objectifs du projet 3

2 Immersion et Prise en Main d'un Projet Existant **3**
2.1 Découverte de l'architecture du Projet et de l'écosystème Julia 4
2.2 Apprentissage du workflow collaboratif (GitHub et VS Code) 4

3 Analyse et Évaluation : Les Profils de Performance **5**
3.1 Concept des profils de performance 5
3.2 Implémentation dans CTBENCHMARK 6
3.3 Résultats et observations 8

4 Amélioration de la Convergence : Résolutions de bugs et Ajustement des Initial Guesses **8**
4.1 Identification des problèmes de convergence 8
4.2 Rôle et modification des Initial Guesses 9
4.3 Comparaison des résultats (avant/après) 10

5 Enrichissement du Benchmark **10**
5.1 Ajout de nouvelles méthodes de discrétisations 10
5.2 Implémentation de nouveaux problèmes 11
5.3 Accélération matérielle : Implémentation d'un benchmark GPU 13
5.4 L'apport de l'IA générative (Gemini CLI) 14

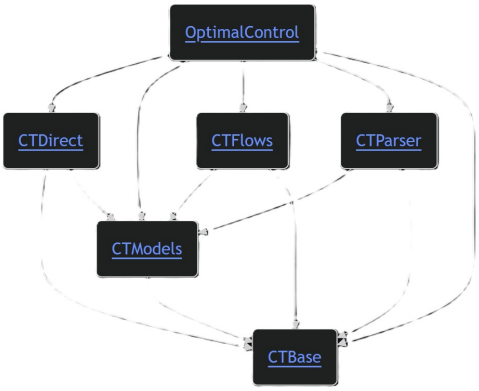
6 Bilan et Conclusion **14**
6.1 Bilan technique 14
6.2 Bilan personnel et compétences acquises 15

1 Introduction

1.1 Contexte global : Control Toolbox

La Control Toolbox est un écosystème open-source, développé principalement dans le langage de programmation Julia, dédié au contrôle mathématique et à ses applications. Son objectif premier est de fournir un ensemble complet d'outils pour modéliser, simuler et résoudre des problèmes de contrôle optimal (OCP - Optimal Control Problems) qui sont régis par des équations différentielles ordinaires.

Cet environnement a une petite spécificité, au lieu d'être un seul bloc unit, la Control Toolbox est divisée en plusieurs paquets spécialisés (comme OptimalControl.jl, CTBase.jl, ou encore CTBenchmarks.jl sur lequel nous avons majoritairement travaillé). Cette structure permet de gérer de manière indépendante chaque étape de l'optimisation, de la formulation mathématique du problème via une syntaxe lisible et intuitive, jusqu'à sa transcription et sa résolution numérique.



La toolbox est conçue pour être à la fois rigoureuse mathématiquement et performante informatiquement. Elle supporte notamment les méthodes de résolution directes et indirectes. Elle permet ainsi d'optimiser des trajectoires, de minimiser des fonctions de coût et de gérer des contraintes dynamiques complexes.

C'est dans cet environnement riche, à la croisée de l'informatique scientifique et des mathématiques appliquées, que s'est inscrit notre projet. Contribuer à un tel écosystème a constitué une mise en pratique idéale des concepts abordés au cours de notre formation en Mathématiques Appliquées.

1.2 Le module CTBenchmarks

Au sein de cet écosystème, le module `CTBenchmarks.jl` joue un rôle fondamental de validation et d'évaluation. En optimisation numérique et en contrôle optimal, il est en effet crucial de pouvoir mesurer et comparer objectivement l'efficacité, la robustesse et la précision des différents algorithmes de résolution (les solveurs).

Pour répondre à ce besoin, `CTBenchmarks.jl` regroupe une bibliothèque standardisée de problèmes de contrôle optimal. Ces problèmes, allant de cas d'école classiques à des modèles plus complexes, servent de cas de test de référence. Ils permettent aux développeurs et aux chercheurs de s'assurer qu'une modification dans la toolbox (ou l'ajout d'un nouveau solveur) n'entraîne pas de régression, et d'évaluer les performances sur des critères précis comme le temps de calcul ou la précision de la convergence.

C'est exactement sur cela que s'est concentré notre travail. Le module nécessitait quelques petits ajouts pour pouvoir évaluer la robustesse des problèmes existants. Notre mission était de fiabiliser les tests existants (notamment ceux souffrant de problèmes de convergence), de la création de nouveaux outils d'analyse de résultats, et de l'intégration de nouveaux problèmes.

1.3 Objectifs du projet

Dans le cadre de ce projet, notre mission principale était de consolider et d'étendre les capacités du module `CTBenchmarks.jl`. Rejoindre une base de code existante et complexe représentait un premier défi, nécessitant une phase d'appropriation de l'architecture et des outils collaboratifs modernes, notamment GitHub.

Plus spécifiquement, le travail de notre binôme s'est articulé autour des axes suivants :

- **L'implémentation de profils de performance :** Il s'agissait dans un premier temps de faire des recherches pour assimiler ce concept d'évaluation (nouveau pour nous en MAM4), puis de le coder afin de pouvoir comparer visuellement et objectivement l'efficacité des différents solveurs.
- **L'optimisation des estimations initiales (*initial guesses*) :** De nombreux problèmes du benchmark ne convergeaient pas ou aboutissaient à de mauvaises solutions (minimums locaux). Notre tâche a consisté à analyser ces cas et à ajuster les *initial guesses* pour guider correctement les algorithmes vers la solution optimale.
- **La résolution de bugs (*bug fixing*) :** Identifier, comprendre et corriger diverses anomalies dans le code pour rendre le module d'évaluation plus robuste.
- **L'ajout de nouveaux problèmes assisté par l'IA :** Enrichir le banc de test en implémentant des problèmes de contrôle optimal inédits. Pour cette phase, nous avons exploré l'utilisation de l'intelligence artificielle, en particulier Gemini CLI, qui nous a fait gagner un temps précieux dans la structuration et la génération du code.

Ce projet s'est donc révélé particulièrement complet, nous permettant d'allier la théorie des mathématiques appliquées et de l'optimisation aux réalités de l'ingénierie logicielle.

2 Immersion et Prise en Main d'un Projet Existant

L'intégration au sein d'un projet *open-source* de l'envergure de la *Control Toolbox* marque une transition significative par rapport aux projets académiques classiques. Avant de pouvoir apporter la moindre contribution à la librairie, il était important de s'approprier l'écosystème, de comprendre son architecture globale et son mécanisme.

2.1 Découverte de l'architecture du Projet et de l'écosystème Julia

La première étape de notre travail a consisté en une phase d'exploration pure. Avant de pouvoir apporter la moindre modification, il a fallu parcourir l'arborescence du dépôt `CTBenchmarks.jl` pour essayer de comprendre sa logique globale. Ce module agissant comme un banc d'essai centralisé pour évaluer les performances de divers solveurs d'optimisation, nous avons pris le temps de repérer les fichiers clés, d'observer comment les problèmes étaient structurés et de saisir le fonctionnement général du projet.

Apprentissage du langage Julia :
Avant ce projet, le langage Julia nous était totalement inconnu. Conçu spécifiquement pour le calcul scientifique et l'optimisation numérique de haute performance. Nous avons dû nous familiariser rapidement avec sa syntaxe. Cela s'est avérée essentiel non seulement pour déchiffrer le code de la *Control Toolbox*, mais également pour écrire nos scripts et implémenter de nouvelles fonctionnalités.

Gestion de l'environnement et reproductibilité :
Il a fallu configurer notre espace de travail. Travailler avec le langage Julia nécessite une gestion stricte des dépendances pour garantir la reproductibilité des résultats d'une machine à l'autre. Notre première action concrète a donc été de cloner le dépôt distant en local et d'initialiser proprement l'environnement de développement, afin de nous assurer que nos futures exécutions s'appuieraient sur les bonnes versions des librairies.

Analyse de la base de code :
Il a fallu décrypter la façon dont les problèmes de contrôle optimal (dynamique, contraintes, fonctions de coût) étaient traduits en structures de données Julia au sein de la *toolbox*. Nous avons cartographié les appels de fonctions entre `CTBenchmarks.jl` et les autres modules de la suite (tels que `OptimalControl.jl` ou `CTBase.jl`) afin d'identifier précisément les points d'injection pour nos futures implémentations (profils de performance, nouveaux problèmes, ajustement des valeurs initiales).

2.2 Apprentissage du workflow collaboratif (GitHub et VS Code)

Pour contribuer efficacement à ce projet, nous avons dû abandonner les méthodes de développement isolées depuis le site Github au profit d'un flux de travail collaboratif assez rigoureux, basé sur Git et la plateforme GitHub. Nous avons paramétré notre environnement de développement intégré (VS Code, couplé à l'extension Julia) pour interagir de manière fluide avec le dépôt distant.

Chaque intervention sur le code, qu'il s'agisse d'une correction de bug ou de l'ajout d'une fonctionnalité, obéissait à ce cycle :

— **Création et traçabilité via les *Issues* :** Aucune ligne de code n'était modifiée sans l'ouverture préalable d'une (*Issue*) sur GitHub. Cette étape permettait de documenter le problème (par exemple, un défaut de convergence sur un problème spécifique) ou la fonctionnalité à implémenter, et d'en discuter la faisabilité et l'approche avec les mainteneurs du projet avant même de commencer à coder.

— **Isolation du travail par création de branches :** Afin de ne jamais compromettre la stabilité de la branche principale (`main`), depuis notre terminal ou l'interface Git de VS Code, nous créions une branche dédiée pour chaque *Issue* (ex : `fix/initial-guess-problem-X` ou `feat/performance-profiles`).

— **Développement itératif et *Commits* atomiques :** Le développement en local se faisait de manière incrémentale. Au lieu de réaliser des sauvegardes massives, nous structurions notre travail sous forme de *commits* atomiques (une modification logique = un *commit*). Cela facilite grandement la lecture de l'historique et la localisation d'éventuelles erreurs.

— **Synchronisation (*Push*) et Intégration Continue (CI) :** Une fois le commit effectué, nous le poussions (*push*) vers le dépôt distant GitHub. Cette action déclenchait automatiquement les *workflows* d'intégration continue (via GitHub Actions). Ces pipelines lançaient la compilation et l'intégralité de la suite de tests unitaires et de *benchmarking* sur des serveurs distants.

— **Revue de code (*Pull Requests*) et Fusion (*Merge*) :** Lorsque l'intégration continue validait notre code, nous ouvrions une *Pull Request* (PR). Ce mécanisme est le cœur de la collaboration *open-source* : il offre un espace de revue (*code review*) où le binôme et les mainteneurs de la *Control Toolbox* pouvaient inspecter nos modifications ligne par ligne, suggérer des optimisations et vérifier le respect des conventions de codage. Une fois la PR approuvée, notre branche était fusionnée (*merge*) avec la branche principale, et clôturait automatiquement l'*Issue* correspondante.

3 Analyse et Évaluation : Les Profils de Performance

3.1 Concept des profils de performance

Lorsqu'il s'agit d'évaluer l'efficacité de la résolution numérique, se limiter à comparer les solveurs d'optimisation n'est pas suffisant. Dans la Control Toolbox, la performance globale dépend fortement de la combinaison entre le model (comme JuMP, ADNLPModels ou ExaModels) et le solveur (comme Ipopt ou MadNLP).

Pour évaluer ces couples, l'utilisation de simples moyennes (comme le temps de calcul moyen) est souvent trompeuse, car une configuration peut être pénalisée par un seul problème extrêmement difficile. Pour faire face à cela, nous avons étudié et implémenté les profils de performance (basés sur les travaux de Dolan et Moré).

L'idée fondamentale est de comparer ces configurations de manière relative. Soit P l'ensemble des problèmes de notre benchmark et C l'ensemble des configurations à évaluer (une configuration $c \in C$ étant définie par un couple `(modèle, solveur)`). Pour chaque problème $p \in P$ et chaque configuration $c \in C$, nous mesurons une valeur de performance $t_{p,c}$ (dans notre cas, le temps CPU).

On définit d'abord le ratio de performance $r_{p,c}$, qui compare la performance de la configuration c sur le problème p par rapport à la meilleure configuration pour ce même problème :

$$r_{p,c} = \frac{t_{p,c}}{\min_{c \in C} t_{p,c}}$$

Ainsi, si le couple `(exa, madnlp)` est le plus rapide pour le problème p, son ratio

vaudra 1. S'il est deux fois plus lent que le meilleur, le ratio vaudra 2.

Le profil de performance d'une configuration c, noté $\rho_c(\tau)$, est ensuite défini comme la fonction de répartition de ces ratios :

$$\rho_c(\tau) = \frac{1}{|P|}\text{card}\{p \in P \mid r_{p,c} \leq \tau\}$$

Concrètement, la valeur $\rho_c(\tau)$ représente la proportion des problèmes pour lesquels les performances de la configuration c sont à un facteur τ près des meilleures performances possibles. L'analyse visuelle de ces courbes (voir Figure 1) nous permet de tirer deux conclusions majeures :

— **L'efficacité (à $\tau = 1$) :** L'ordonnée à l'origine de la courbe indique le pourcentage de problèmes pour lesquels le couple `(modèle, solveur)` a été strictement le plus rapide.

— **La robustesse (lorsque $\tau \to \infty$) :** L'asymptote horizontale de la courbe indique la proportion totale de problèmes que la configuration a réussi à résoudre (convergence réussie), indépendamment du temps pris.

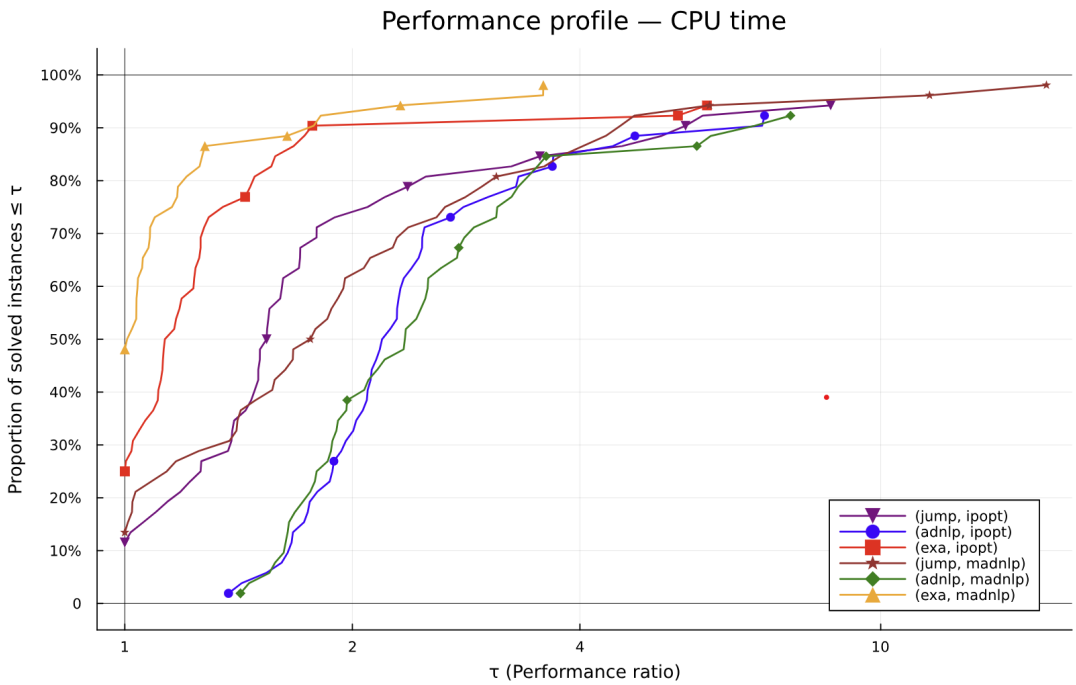


FIGURE 1 : Profil de performance sur le temps CPU généré via CTBenchmarks.jl.

3.2 Implémentation dans CTBENCHMARK

L'implémentation des profils de performance au sein de `CTBenchmarks.jl` a nécessité de traduire le formalisme mathématique en langage Julia. L'intégralité du code source étant disponible sur le dépôt GitHub du projet, nous détaillons ici les choix d'implémentation majeurs.

Notre développement se déroule autour de quatre piliers principaux :

1. Une architecture orientée données (DataFrames) Pour manipuler efficacement les résultats des différents solveurs, nous avons choisi de nous appuyer sur le package `DataFrames.jl`. Les données brutes, issues de fichiers JSON générés par les benchmarks, sont chargées dans un DataFrame (`load_benchmark_df`). Cela nous permet d'utiliser des opérations très utiles (`filter`, `groupby`, `combine`) pour trier, filtrer les échecs de convergence et agréger les temps de calcul.

2. Modularité des critères d'évaluation Afin de rendre l'outil générique, nous avons créé des structures de données spécifiques, notamment `ProfileCriterion` et `PerformanceProfileConfig`. Cette approche permet de générer des profils non seulement sur le temps CPU, mais sur n'importe quelle autre donnée (nombre d'itérations, évaluations de fonctions, etc.) sans modifier le cœur de l'algorithme.

3. Calcul des ratios de Dolan-Moré La fonction centrale de notre module, `_compute_dolan_more_ratios`, se charge de confronter chaque configuration (le couple modèle/solveur) à la meilleure configuration disponible pour un problème donné. L'extrait ci-dessous illustre la logique implémentée :

```julia
function _compute_dolan_more_ratios(df, cfg)
    # Regroupement par probleme pour trouver la meilleure metrique
    inst_grouped = groupby(df, cfg.instance_cols)

    # Calcul de la meilleure performance (best_metric) pour chaque probleme
    df_best = combine(inst_grouped, :metric ⇒ _best_metric ⇒ :best_metric)
    df_with_best = leftjoin(df, df_best; on=cfg.instance_cols)

    # Calcul du ratio de performance (r = temps / meilleur_temps)
    ratios = Float64[]
    for i in 1:nrow(df_with_best)
        m = df_with_best.metric[i]
        b = df_with_best.best_metric[i]

        # Gestion des cas d'echec de tous les solveurs ou d'egalite parfaite
        r = isinf(b) ? NaN : (m == b ? 1.0 : m / b)
        push!(ratios, r)
    end
    df_with_best.ratio = ratios
    return df_with_best
end
```

Extrait : Logique de calcul des ratios dans CTBenchmarks.jl

4. Visualisation et analyse statistique automatisée Enfin, la restitution des résultats se fait sous deux formes. Visuellement, la fonction `plot_performance_profile` s'appuie sur le package `Plots.jl` pour générer les courbes de répartition (avec placement intelligent des marqueurs en échelle logarithmique). Analytiquement, nous avons implémenté une fonction `analyze_performance_profile` qui génère automatiquement un rapport textuel synthétisant l'efficacité et la robustesse (taux de résolution) de chaque combinaison.

3.3 Résultats et observations

La Figure 1 présente les profils de performance obtenus sur un ensemble de 52 instances de problèmes de contrôle optimal, en utilisant le temps de calcul (CPU time) comme critère de comparaison. Ces profils évaluent l'efficacité et la robustesse de six couples modèle-solveur en illustrant la proportion de problèmes résolus en fonction du ratio de temps de calcul par rapport au couple le plus rapide (τ). Au total, 98,1 % des instances ont été résolues avec succès par au moins un couple.

Efficacité (Rapidité absolue) : L'analyse à l'origine du graphique ($\tau = 1$) permet d'évaluer l'efficacité pure des approches. On observe que le couple **(exa, madnlp)** domine très nettement en termes de vitesse absolue, s'imposant comme le plus rapide sur **48,1 %** des instances. En deuxième position, le couple **(exa, ipopt)** n'est strictement le meilleur que sur **25,0 %** des problèmes. On remarque également que les formulations basées sur le modèle `adnlp` ne sont jamais les plus rapides (0,0 % d'efficacité pure), quel que soit le solveur utilisé. Cela souligne l'impact majeur du choix du backend de différenciation sur la rapidité de résolution.

Robustesse (Fiabilité globale) : L'observation du comportement asymptotique des courbes (lorsque τ devient grand) met en évidence la robustesse des méthodes. Deux couples se distinguent comme étant les plus fiables à égalité : **(exa, madnlp)** et **(jump, madnlp)**, parvenant tous deux à résoudre **98,1 %** de l'ensemble des problèmes. À l'inverse, les modèles modélisés avec `adnlp` (couplés à `ipopt` ou `madnlp`) sont les moins robustes de la sélection, plafonnant à 92,3 % de réussite (soit un taux d'échec de **7,7 %**).

Observations spécifiques : Il est intéressant de noter qu'une instance s'est révélée particulièrement difficile : le problème `space_shuttle` (avec une taille de grille $N = 2000$). C'est la seule instance du panel de test (sur les 52) qui n'a convergé avec aucun des six couples modèle-solveur, expliquant pourquoi la robustesse maximale des meilleurs solveurs plafonne à 98,1 % au lieu de 100 %.

4 Amélioration de la Convergence : Résolutions de bugs et Ajustement des Initial Guesses

4.1 Identification des problèmes de convergence

Grâce à la suite de tests automatisée `CTBenchmarks.jl`, nous avons pu identifier des causes de bugs et des problèmes de convergence qui entravaient la résolution de certains modèles de contrôle optimal :

Résultats non disponibles (NA) :
Dans de nombreuses configurations, les tableaux de résultats affichaient la mention "NA" (ou des valeurs manquantes), signalant un échec total du solveur avant même d'avoir pu produire une première itération valide.

Objectifs non respectés :
Pour certains problèmes, bien que le solveur indique une convergence apparente, la valeur de l'objectif final était totalement aberrante ou incohérente avec la physique du système (ex : une altitude négative ou une consommation d'énergie infinie).

Temps de résolution aberrants :
Nous avons observé des temps de calcul disproportionnés pour des problèmes qui, une fois bien initialisés, se résolvent en quelques millisecondes. Ce comportement de "stagnation" du solveur est typique d'un point de départ trop éloigné de la solution.

Incohérences de définition (Min/Max) :
Une source majeure d'erreur provenait de la définition même du sens de l'optimisation à la fin du problème. Des confusions entre la minimisation d'un coût et la maximisation d'un critère (ex : maximiser l'altitude finale vs minimiser l'opposé de l'altitude).

Limites de temps et d'itérations :
Les benchmarks ont montré que des problèmes comme glider ou space shuttle atteignaient souvent le max_iter ou le max_wall_time sans converger, particulièrement lors de l'augmentation de la taille de la grille ($grid_size$).

Instabilité de la discrétisation :
L'analyse comparative entre les méthodes de discrétisation trapeze et midpoint a souligné que certains problèmes sont extrêmement sensibles au point de départ pour satisfaire les contraintes de dynamique dès les premières itérations.

4.2 Rôle et modification des Initial Guesses

Pour corriger ces anomalies, nous avons agi sur deux leviers : la cohérence des modèles et la qualité du point de départ.

Ajustement par l'objectif (Refining by Target) :
Notre méthode consistait souvent à analyser la valeur de l'objectif obtenue lors d'une première tentative (même partielle) pour ensuite réajuster manuellement l'initial guess (init). En modifiant les valeurs initiales pour qu'elles se rapprochent de l'objectif attendu ou d'une solution physiquement réaliste, nous avons permis au solveur de rester dans le "bassin d'attraction" de l'optimum global.

Standardisation des critères (Min/Max) :
Nous avons rigoureusement vérifié et harmonisé les définitions d'objectifs pour chaque backend (JuMP, OptimalControl). Cette étape a permis d'éliminer les erreurs de signe qui rendaient les problèmes insolubles.

Élimination des singularités :
L'ajustement des estimations initiales a permis de maintenir les variables hors des zones critiques où les fonctions (comme sincos ou les divisions par la masse) génèrent des erreurs numériques fatales.

4.3 Comparaison des résultats (avant/après)

L'impact de ces ajustements est clairement visible dans les rapports de CTBenchmarks.jl :

Disparition des "NA" :
Les instances de test qui échouaient systématiquement produisent désormais des solutions valides et vérifiables.

Respect de l'optimum :
Les valeurs d'objectif sont maintenant cohérentes entre les différents solveurs (Ipopt et MadNLP) et les différents modèles, confirmant que le sens d'optimisation (min/max) est correctement interprété.

Normalisation des temps :
Les temps de résolution "aberrants" ont été remplacés par des performances industrielles, avec une convergence rapide même pour des discrétisations fines (*gridsize* élevé), améliorant ainsi l'efficacité globale de la chaîne de calcul.

5 Enrichissement du Benchmark

5.1 Ajout de nouvelles méthodes de discrétisations

Une fois les outils d'analyse de performances mis en place, notre mission a consisté à concevoir et exécuter de nouveaux benchmarks pour évaluer des aspects spécifiques. L'un des scénarios majeurs que nous avons implémenté est la comparaison des méthodes de discrétisation temporelle.

L'objectif de ce benchmark était d'évaluer l'impact du choix du schéma d'intégration numérique : en outre la méthode du point milieu (`:midpoint`) face à la méthode des trapèzes (`:trapeze`) sur l'efficacité des algorithmes d'optimisation.

Pour ce faire, nous avons automatisé une série de tests confrontant ces deux discrétisations à travers l'interface `CTBenchmarks.jl`. Nous avons ciblé une grille de discrétisation fixe ($N = 200$) et utilisé le backend de différentiation `ExaModels` couplé aux solveurs `Ipopt` et `MadNLP`.

L'extrait de code ci-dessous illustre la configuration de ce scénario :

```
function run_discretization_benchmark()
    results = CTBenchmarks.benchmark(;
        solver_models = [
            :madnlp ⇒ [:exa],
            :ipopt  ⇒ [:exa],
        ],
        disc_methods = [:trapeze, :midpoint],
        grid_sizes = [200],
        tol = 1e-8,
        ipopt_mu_strategy = "adaptive",
```

```
        print_trace = false,
        max_iter = 2000,
        max_wall_time = 600.0,
    )
    println(" Benchmark midpoint vs trapeze completed successfully!")
    return results
end
```

Extrait : Configuration du benchmark comparatif des discrétisations

La création de ce script permet de standardiser l'évaluation : en fixant des critères d'arrêt stricts (tolérance de 10^{-8}, limite de temps de 600 secondes et un maximum de 2000 itérations), nous garantissons l'équité de la comparaison.

Les données générées par l'exécution de cette fonction (`results`) sont ensuite directement exploitables par notre module de profils de performance, permettant d'obtenir un diagnostic visuel et chiffré (taux de succès, temps de résolution) départageant la méthode des trapèzes et celle du point milieu sur l'ensemble des problèmes de la bibliothèque.

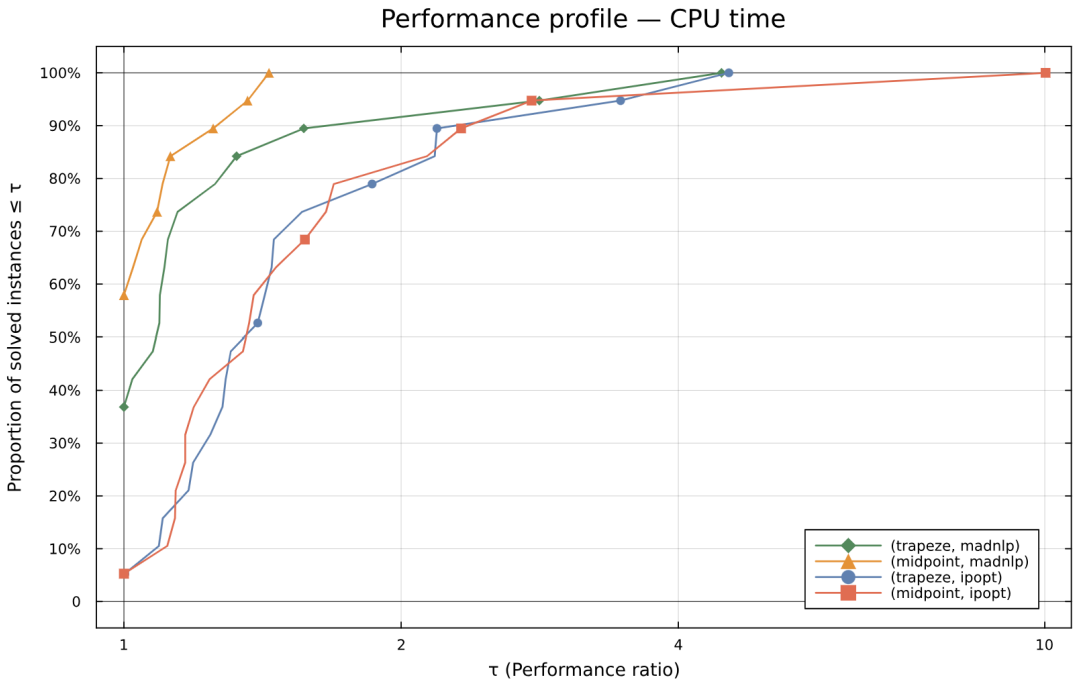


FIGURE 2 : Profil de performance midpoint vs trapeze

5.2 Implémentation de nouveaux problèmes

Par la suite, nous avons voulu implémenter quelques problèmes, nous en avons sélectionné quatre aux caractéristiques mathématiques variées :

— **Le problème du Brachistochrone :** Un problème historique fondamental consistant à déterminer la trajectoire minimisant le temps de glissement d'une masse soumise à la gravité entre deux points.

— **Le problème de Bryson-Denham :** Un problème canonique d'optimisation sous contraintes d'état, particulièrement utile pour tester la manière dont les solveurs gèrent les bornes strictes sur la trajectoire.
— **Le problème du Mountain Car :** Un grand classique (souvent utilisé en apprentissage par renforcement) où un véhicule sous-motorisé doit balancer d'avant en arrière pour accumuler l'énergie potentielle nécessaire afin de gravir une pente raide.
— **Le problème du Balanced Field (décollage) :** Un problème d'ingénierie aérospatiale complexe modélisant le décollage d'un avion, incluant des contraintes opérationnelles comme la distance critique d'accélération-arrêt.

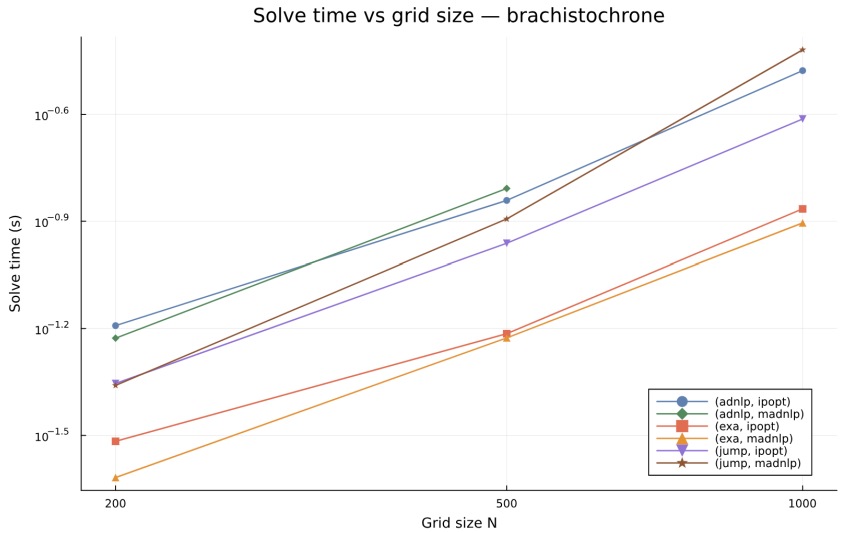


FIGURE 3 : Temps vs Grid Size pour le problème brachistochrone

Méthodologie et implémentation La première étape a consisté à réunir les formulations mathématiques rigoureuses de ces problèmes. Pour ce faire, nous nous sommes appuyés sur la documentation de *Dymos* (un outil d'optimisation de trajectoire développé par la NASA via OpenMDAO), qui répertorie l'écriture mathématique précise (dynamiques, fonctions de coût, conditions aux limites et contraintes) de ces cas d'étude.

Une fois les modèles mathématiques définis, il a fallu les traduire en langage Julia. L'écosystème de la *Control Toolbox* possédant une syntaxe déclarative très spécifique pour définir les Problèmes de Contrôle Optimal (OCP), nous avons adopté une démarche par mimétisme (*reverse engineering*).

Nous avons étudié en détail le code source des problèmes déjà implémentés dans le dépôt GitHub `OptimalControlProblems.jl`, en prenant notamment comme modèle de référence le problème de la poutre (`beam`). Ce calquage nous a permis de comprendre comment structurer notre code : déclaration de l'état et du contrôle, définition de la dynamique (`x'(t) = ...`), des conditions de Mayer ou de Lagrange, et gestion des paramètres. Ainsi, nous nous sommes assurés que nos quatre nouveaux problèmes s'intégraient nativement et parfaitement dans l'architecture de la toolbox, prêts à être appelés par les solveurs du benchmark.

5.3 Accélération matérielle : Implémentation d'un benchmark GPU

La résolution des grands systèmes linéaires (KKT) constituant l'opération la plus coûteuse de l'optimisation non linéaire, nous avons voulu évaluer le gain potentiel de l'accélération matérielle. Pour ce faire, nous avons implémenté une nouvelle suite de tests spécifiquement dédiée, baptisée `core-kkt-gpu`.

Protocole expérimental Notre jeu de données s'est concentré sur les mêmes problèmes de contrôle optimal qu'avant. En faisant varier la taille des grilles de discrétisation, nous avons généré un total de 76 instances d'évaluation. Sur cet ensemble, nous avons confronté deux configurations basées sur le solveur `MadNLP` couplé au model `ExaModels` :

— L'exécution standard sur processeur : `(exa, madnlp)`.

— L'exécution déportant les calculs matriciels sur la carte graphique : `(exa_gpu, madnlp)`.

Analyse statistique des résultats Les données extraites par notre module d'analyse révèlent un taux de succès global très satisfaisant, avec 149 exécutions convergées sur 152 (soit $98,0\,\%$). De plus, les profils de performance nous ont permis de quantifier précisément le compromis entre stabilité et vitesse :

— **Robustesse (taux de résolution) :** La version CPU classique s'est montrée marginalement plus stable, parvenant à trouver l'optimum sur $98,7\,\%$ des instances, contre $97,4\,\%$ pour la version GPU.

— **Efficacité (rapidité relative) :** C'est sur le critère du temps de calcul que l'architecture GPU prend l'avantage. La configuration `(exa_gpu, madnlp)` s'est imposée comme la méthode strictement la plus rapide sur $57,9\,\%$ des instances, reléguant le CPU à $42,1\,\%$.

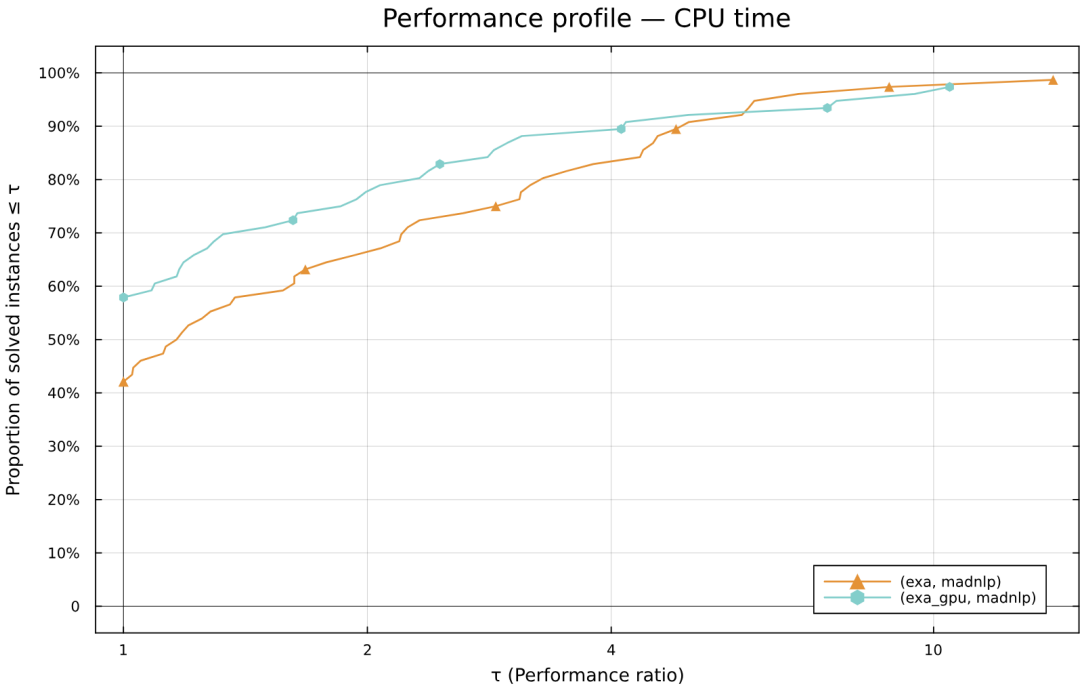


FIGURE 4 : Profil de performance CPU vs GPU

En résumé, ces résultats confirment que le GPU est un vrai atout pour la *Control Toolbox*. La très légère baisse de robustesse (souvent liée aux échanges de données entre le processeur et la carte graphique) est largement compensée par le gain de vitesse. Le fait de pouvoir calculer les matrices en parallèle réduit considérablement le temps d'exécution. Ainsi, plus la taille des problèmes augmente, plus l'utilisation du GPU devient intéressante pour accélérer la résolution.

5.4 L'apport de l'IA générative (Gemini CLI)

Lors de ce projet, nous avons eu l'opportunité d'intégrer un outil devenu incontournable : l'intelligence artificielle. Dans le contexte actuel, il nous semblait non seulement naturel, mais presque indispensable d'apprendre à nous en servir concrètement sur une base de code réelle de grande envergure.

Dans un premier temps, nous avons utilisé l'interface web classique de Gemini. Cependant, cette approche a rapidement montré ses limites. L'IA n'ayant pas accès au projet dans son entièreté, elle peinait à s'imprégner de l'environnement de la *Control Toolbox*.

Travaillant quotidiennement sur l'éditeur VSCode, nous avons cherché une solution capable de pallier ce manque de contexte. Notre choix s'est naturellement porté sur **Gemini CLI**. Directement intégré à note éditeur, cet outil s'est imposé comme une évidence.

Il s'est révélé être un atout majeur, nous accompagnant efficacement dans l'écriture de la syntaxe des nouveaux problèmes et nous faisant gagner un temps précieux lors des phases de résolution de bugs. Cette expérience a transformé notre méthode de travail : aujourd'hui, il nous paraît bien difficile de nous séparer d'un tel assistant lorsqu'une erreur inattendue fait son apparition.

6 Bilan et Conclusion

6.1 Bilan technique

D'un point de vue technique, notre contribution a permis de consolider et d'enrichir le module `CTBENCHMARK` de la *Control Toolbox*.

Tout d'abord, l'implémentation des profils de performance dote désormais la toolbox d'un véritable outil d'évaluation standardisé et automatisé. Il est aujourd'hui possible d'analyser visuellement et statistiquement l'efficacité et la robustesse des différents couples solveur/modèle de manière rigoureuse.

Ensuite, la mise en place de scénarios de tests automatisés, tels que la comparaison systématique des schémas de discrétisation (`midpoint` face à `trapeze`), permet désormais d'orienter les futurs choix d'implémentation des utilisateurs sur la base de métriques concrètes (temps de calcul, taux de convergence,...).

Enfin, l'élargissement de la bibliothèque avec quatre nouveaux problèmes de référence (*Brachistochrone*, *Bryson-Denham*, *Mountain Car*, *Balanced Field*) augmente considérablement la couverture de tests du benchmark. Ces ajouts garantissent que les algorithmes d'optimisation actuels et futurs seront confrontés à une plus grande diversité de contraintes (bornes sur l'état, temps libre, etc.) et de dynamiques non linéaires.

Au final, le module d'évaluation est ressorti de ce projet plus complet, plus fiable et prêt à supporter les futurs développements de l'écosystème Julia.

6.2 Bilan personnel et compétences acquises

Au-delà des apports techniques, ce projet a représenté une étape charnière dans notre formation d'ingénieurs en mathématiques appliquées. Il nous a confrontés aux réalités et aux exigences du développement logiciel au sein d'un écosystème complexe et préexistant.

Cette expérience nous a permis de développer et de consolider un ensemble de compétences à la fois techniques et méthodologiques :

— **Immersion et collaboration sur un code existant :** L'un des plus grands défis a été de lire, comprendre et nous approprier le code écrit par d'autres développeurs. L'utilisation quotidienne de **GitHub** nous a formés aux standards du travail collaboratif moderne (gestion des versions, branches, suivi des modifications), indispensables pour s'intégrer à un projet open source de cette envergure.

— **Montée en compétences en optimisation numérique :** Traduire des concepts mathématiques théoriques (comme les profils de performance de Dolan-Moré) en algorithmes fonctionnels a renforcé notre compréhension du contrôle optimal. Nous avons saisi l'importance cruciale de la formulation des problèmes (choix de l'estimation initiale, méthodes de transcription temporelle) sur le comportement des solveurs NLP.

— **Maîtrise du langage Julia :** Ce projet nous a permis de monter en puissance sur Julia, un langage particulièrement adapté au calcul scientifique, en exploitant des bibliothèques phares de son écosystème telles que `DataFrames.jl` et `Plots.jl`.

— **Intégration d'outils d'IA dans le flux de travail :** Comme détaillé précédemment, l'utilisation de **Gemini CLI** nous a appris à formuler des requêtes précises (*prompt engineering*) pour interagir avec une base de code existante. Savoir utiliser l'IA comme un assistant de développement et de débogage est aujourd'hui une compétence à part entière que nous avons pu expérimenter concrètement.

En conclusion, ce projet fut extrêmement formateur. Il a créé un pont direct entre la théorie mathématique enseignée dans notre cursus et la réalité de l'ingénierie logicielle scientifique, nous préparant efficacement à nos futurs défis professionnels.